

SkateGuard Final Engineering Report

Abstract

SkateGuard is an embedded skateboard monitoring system combining inertial sensing, GPS, local wireless communication, haptic feedback and a board hosted web dashboard. The system uses a Seeed XIAO ESP32 C3 as the main controller, a BNO055 IMU for orientation and rotational sensing, a NEO 8M GPS for speed and route information, and a second XIAO ESP32 C3 wrist unit for display and alerts. The final design performs sensing, event detection and storage on the board while providing rider information through ESP NOW and a local web interface.

1. System Requirements and Final Architecture

The board is the primary sensing and decision making unit. The phone is an interface connected through the rider's hotspot, while the wrist unit provides immediate rider feedback. No cloud database is required for the core telemetry path. The board hosts the web application and stores session data locally.

The board does the work; the phone and the wrist only show it

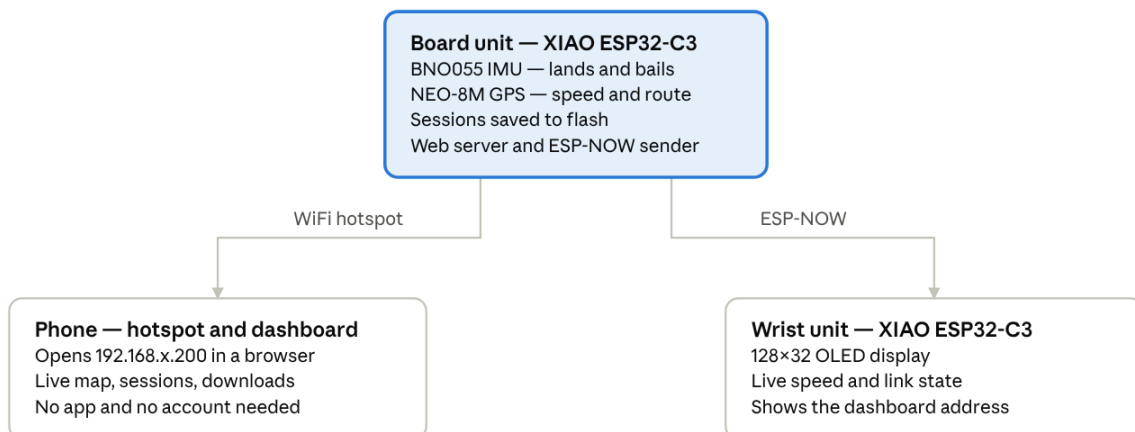


Figure 1. SkateGuard system architecture: board unit, phone dashboard and wrist unit.

1.1 Final Hardware Configuration

Parameter	Value	Purpose / implementation
Board MCU	Seeed XIAO ESP32 C3	Primary sensing, processing, storage and wireless controller
Board IMU	Adafruit BNO055	Fused orientation and gyro data
Board GPS	NEO 8M	Position, satellite count, speed and distance
Board battery	1500 mAh lithium cell	Portable power
Board battery sense	GPIO4, 30k / 10k divider	Voltage monitoring
Wrist MCU	Seeed XIAO ESP32 C3	Telemetry receiver and rider interface
Wrist display	SSD1306 128 x 32 OLED	Live status display

Parameter	Value	Purpose / implementation
Wrist motor	GPIO3	Haptic feedback
Wrist LED	GPIO8	Visual feedback
Wrist battery	1500 mAh lithium cell	Portable power
Wireless	ESP NOW	Board to wrist telemetry
Phone interface	Local HTTP dashboard	Live telemetry and session control

2. Electrical and Interface Details

2.1 Pin and Interface Map

Parameter	Value	Purpose / implementation
Board BNO055 SDA	GPIO6 / D4	I2C data, address 0x28
Board BNO055 SCL	GPIO7 / D5	I2C clock
Board GPS RX	GPIO20	UART1 receive
Board GPS TX	GPIO21	UART1 transmit
Board battery ADC	GPIO4 / ADC1_CH4	30k / 10k divider
Wrist OLED SDA	GPIO6 / D4	I2C data
Wrist OLED SCL	GPIO7 / D5	I2C clock
Wrist motor	GPIO3	Haptic output
Wrist LED	GPIO8	Status output
Wrist battery ADC	GPIO2 / D0	30k / 10k divider

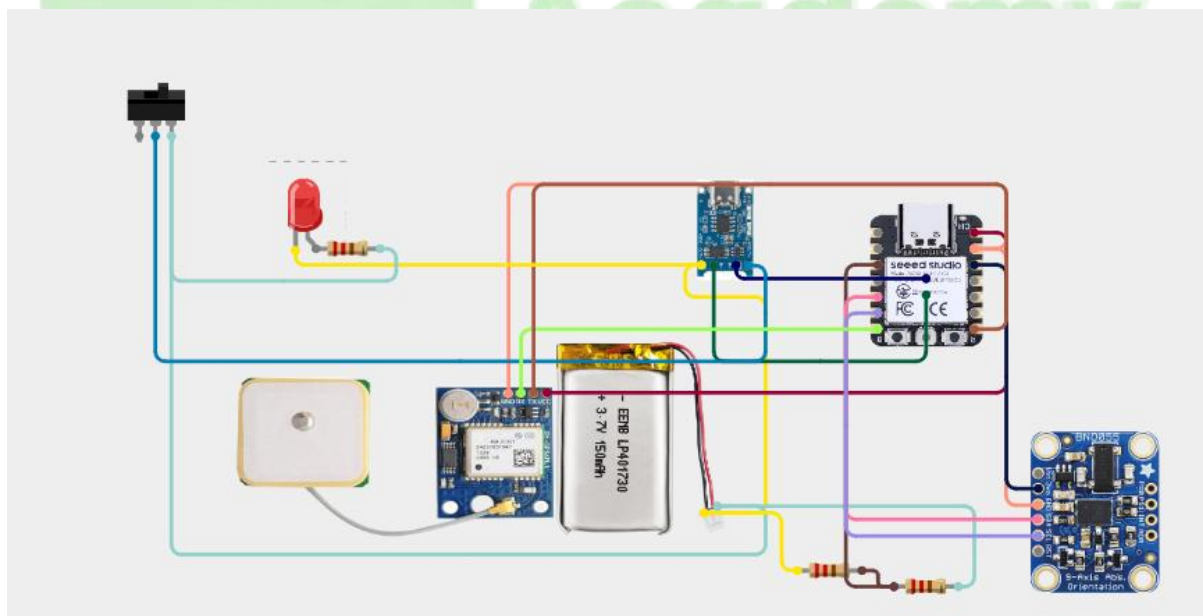


Figure 2. Board unit circuit diagram.

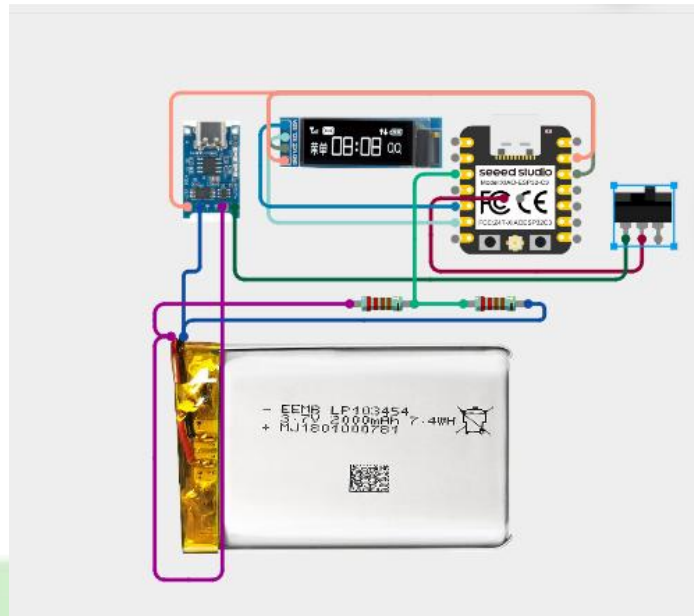


Figure 3. Wrist unit circuit diagram.

3. Mechanical Design

The board enclosure is a two part printed structure mounted around the truck baseplate. The GPS antenna faces upward, the BNO055 is mounted in a dedicated pod at the deck edge, and the XIAO, cell and charging hardware occupy the remaining compartment. The wrist enclosure uses four strap lugs, an OLED window, indicator openings and a recessed USB C charging port. The source project report notes that print material, layer height, infill and final dimensions still need to be recorded.

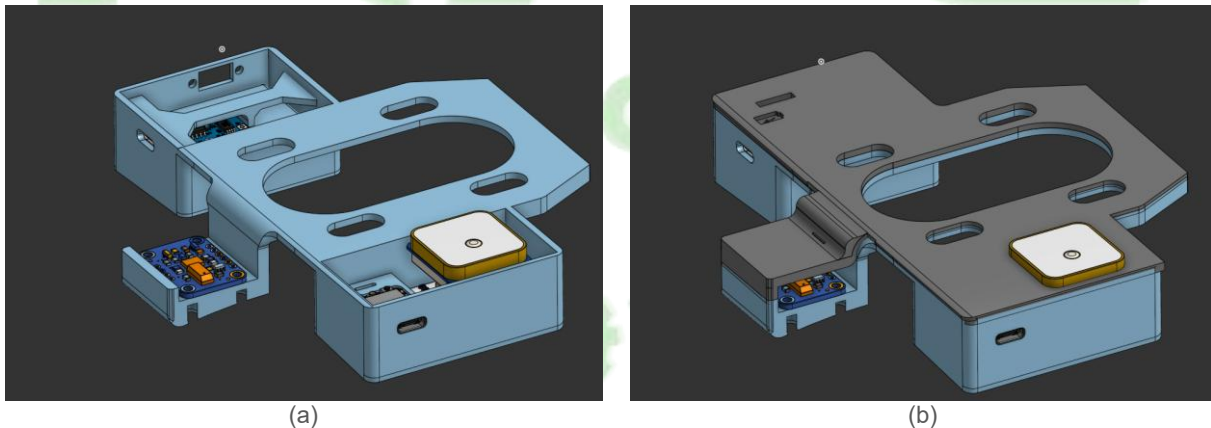


Figure 4. Board enclosure CAD around the truck baseplate: (a) internal layout with IMU pod and GPS, (b) closed assembly.

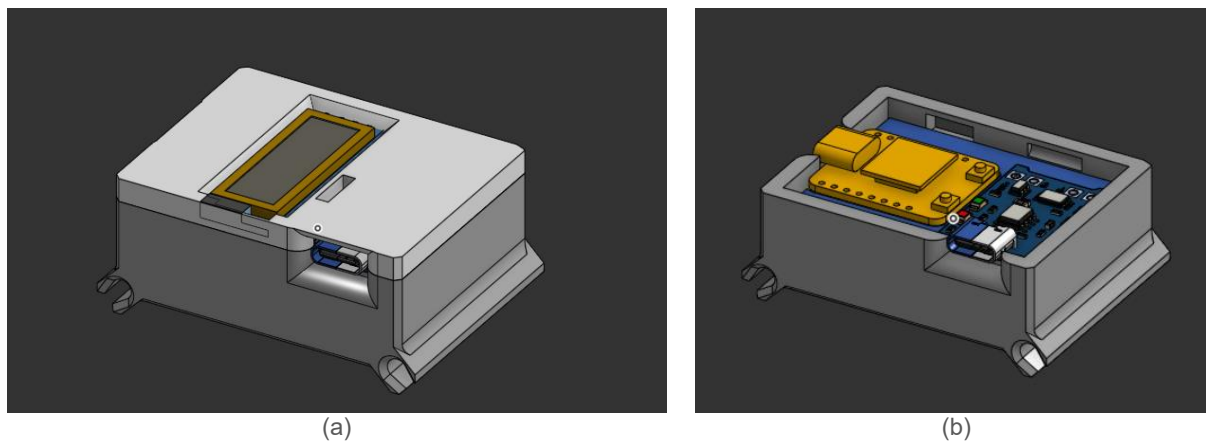


Figure 5. Wrist enclosure CAD: (a) OLED display window and strap lugs, (b) controller and charging module placement.

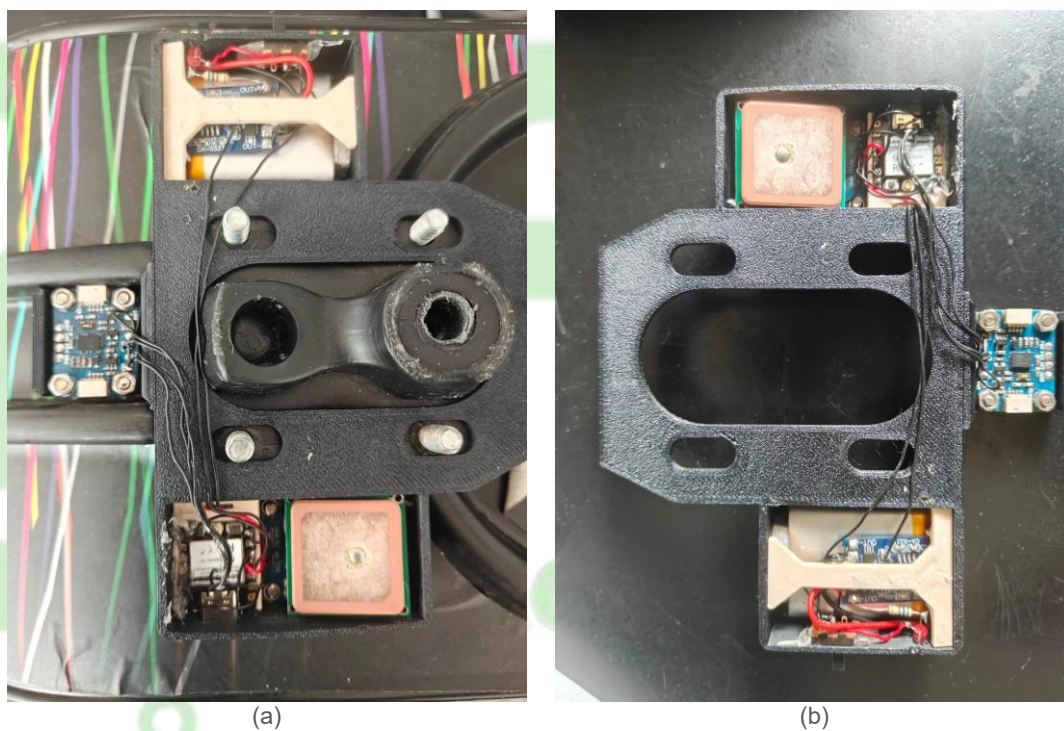


Figure 6. Assembled board unit mounted around the truck baseplate: (a) installed view, (b) top view of the enclosure.

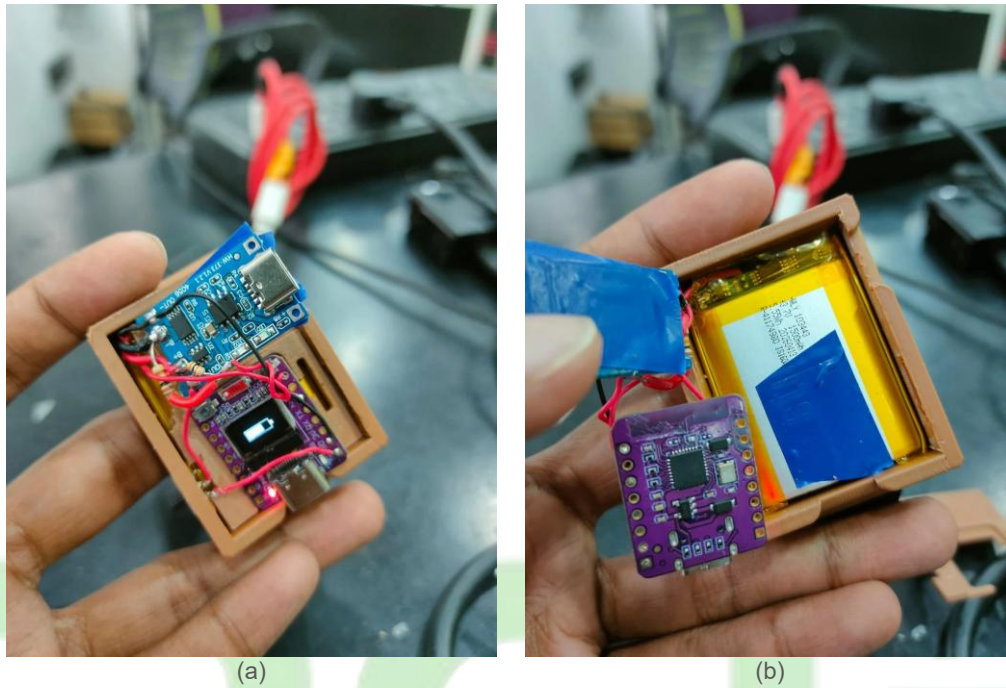


Figure 7. Assembled wrist unit: (a) controller, OLED and charging hardware, (b) battery installation.

4. Firmware Architecture

The firmware combines IMU sampling, GPS parsing, battery measurement, motion state processing, session recording, local HTTP services and ESP NOW telemetry. The board sends telemetry every 100 ms. The dashboard requests live telemetry every 500 ms. Motion stage information is logged every 500 ms.

4.1 Firmware Timing and Storage Parameters

Parameter	Value	Purpose / implementation
ESP NOW telemetry	100 ms	Board to wrist telemetry period
Dashboard polling	500 ms	Live HTTP update period
Stage log	500 ms	Motion state logging period
Stage log capacity	2000 rows	Stored motion history limit
Session storage	7 sessions	Saved ride limit
Route sampling	30 s	Stored route point interval
Route points	20/session	Maximum route points
Program partition	91% used of 1.25 MB	Current firmware size
RAM	12%	Reported runtime memory usage

5. Kickflip Detection Algorithm

Kickflip detection uses a four stage finite state machine. The current firmware assigns GYRO_Y as the kickflip axis. Rotation is accumulated during the active motion window. When sufficient rotation is reached, the detector enters LAND_CONFIRM and observes the settling swing. The algorithm therefore distinguishes the rotational event from the landing response.

A flip counts as a land only if the deck settles within 83 degrees

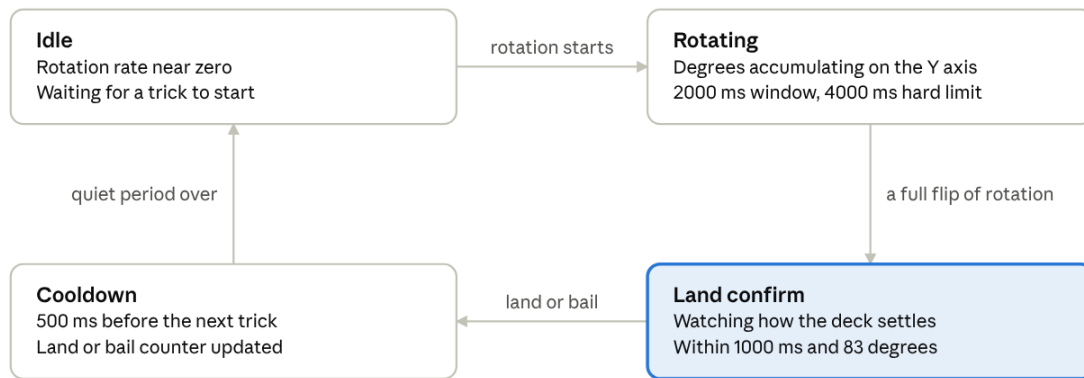


Figure 8. Kickflip detection state machine: Idle, Rotating, Land confirm and Cooldown.

5.1 Kickflip Detection Constants

Parameter	Value	Purpose / implementation
MOTION_TRIGGER_DPS	150 deg/s	Starts rotational detection
MOTION_WINDOW_MS	2000 ms	Active rotation window
MOTION_ABSOLUTE_TIMEOUT_MS	4000 ms	Hard timeout
LAND_CONFIRM_WINDOW_MS	1000 ms	Landing confirmation interval
LAND_CONFIRM_MAX_DEG	83 deg	Maximum settling swing
MOTION_COOLDOWN_MS	500 ms	Post event quiet interval
WOBBLE_OSCILLATIONS_REQUIRED	3	Wobble confirmation count
WOBBLE_REVERSAL_SAMPLES_REQUIRED	5	Required reversal samples
WOBBLE_MAX_CONSIDER_DEG	60 deg	Wobble consideration range
WOBBLE_MIN_SPEED_KMH	5 km/h	Minimum speed for wobble logic

6. GPS Processing and Distance Tracking

The GPS layer rejects stale or incomplete positions. A position must be valid, younger than the configured age limit and supported by at least four satellites. The firmware rejects very small speeds to prevent stationary drift and rejects implausible jumps. Distance is accumulated only from accepted positions, which prevents a single bad GPS fix from corrupting the ride total.

7. Wobble Detection

Wobble is not triggered by the original 25 km/h threshold. The final logic looks for repeated directional reversals in the selected gyro axis. Three counted oscillations are required before the event is reported. This is intended to make the detected signal the physical wobble pattern rather than a GPS speed condition.

8. Wireless Communication and Haptic Feedback

ESP NOW is used for board to wrist telemetry. The board registers a broadcast peer on channel 0 so that the station interface channel is followed. The wrist scans channels at startup until it receives a valid packet, then retunes to the board channel. The telemetry contains speed, distance, tilt angle, board IP, GPS fix state, wobble state, kickflip state and channel information. The wrist motor and LED respond to confirmed wobble events without blocking the main control path.

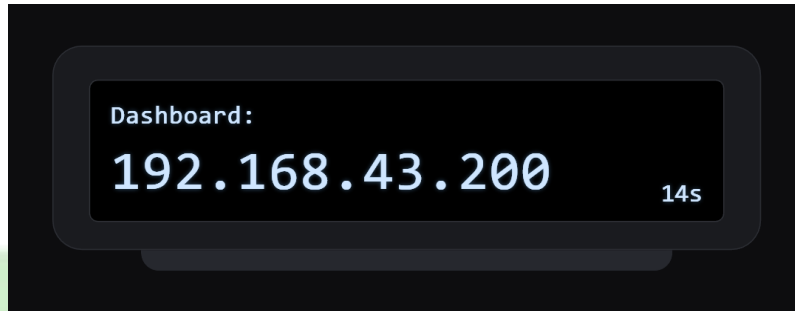


Figure 9. Wrist unit OLED showing the board dashboard address received over ESP NOW.

9. Bail Detection

The planned impact spike method was replaced by settling based classification. A clean landing may produce a high impact while a bail may not. The final method evaluates the post rotation swing and uses the 83 degree and 1000 ms confirmation conditions to classify the outcome.

10. Dashboard and Local Data Architecture

The board hosts the complete dashboard using an ESP32 WebServer. The phone accesses the board over the local hotspot. Core telemetry and commands therefore remain local. The dashboard exposes live telemetry, debug information, sessions, commands and motion log access. Saved sessions contain ride statistics and route data. The last seven rides are retained.

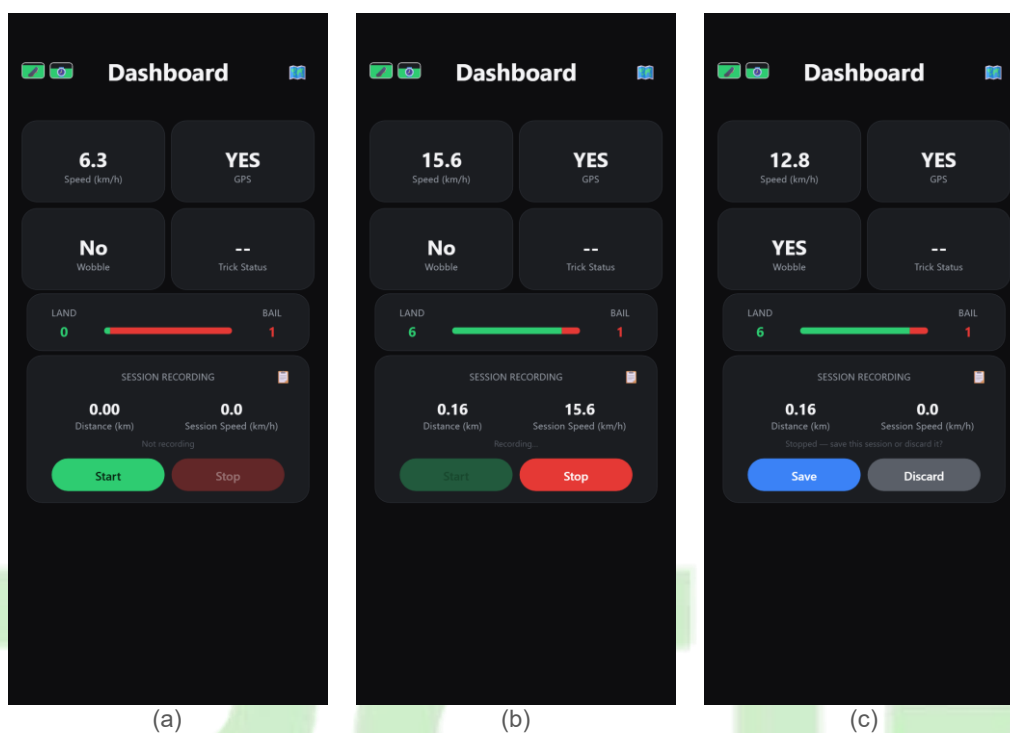


Figure 10. Board hosted dashboard: (a) live telemetry, (b) session recording, (c) Save or Discard after a ride.



Figure 11. Recorded sessions list showing distance, average speed and land/bail counts (last 7).



Figure 12. Debug page showing motion stage, timing windows, counters, GPS, radio and memory diagnostics with stage log download.

10.1 HTTP API

Parameter	Value	Purpose / implementation
/	Dashboard	Main rider interface
/api/live	Telemetry JSON	Live board status
/api/debug	Diagnostic snapshot	Motion, GPS, radio and memory state
/api/sessions	Saved sessions	Ride list and route information
/api/command?cmd=	Session commands	Start, stop, save, discard and delete
/log	Motion log	Download stage log
/clear	Log maintenance	Clear motion log
/manifest.json	Web app manifest	Add to Home Screen

11. Development Changes and Engineering Rationale

MPU6050 to BNO055: The BNO055 provides onboard sensor fusion and direct orientation output.

BLE plus companion app to ESP NOW plus self hosted dashboard: The final design avoids application installation and keeps the core telemetry path local.

25 km/h wobble gate to oscillation detection: GPS speed depends on fix quality and can lag movement, while the oscillation is the physical symptom being detected.

Impact spike bail detection to settling based detection: Impact alone does not reliably separate a landing from a bail.

12. Testing Strategy and Results

12.1 System Test Matrix

Parameter	Value	Purpose / implementation
IMU	Gyro and filtered response	Stabilization behavior visualized
Kickflip	Stage log and accumulated rotation	State machine behavior recorded
GPS	Satellite count, fix age and outdoor route	Outdoor fix obtained with 4+ satellites
Wireless	Startup order, range and channel changes	Wrist unit re finds board without reboot
Dashboard	Save, discard, delete, cap and downloads	Workflows exercised; route issue reproduced and fixed
Field use	Real rides and route maps	Routes rendered over map tiles
Detection accuracy	Real ride comparison	Not yet quantitatively established

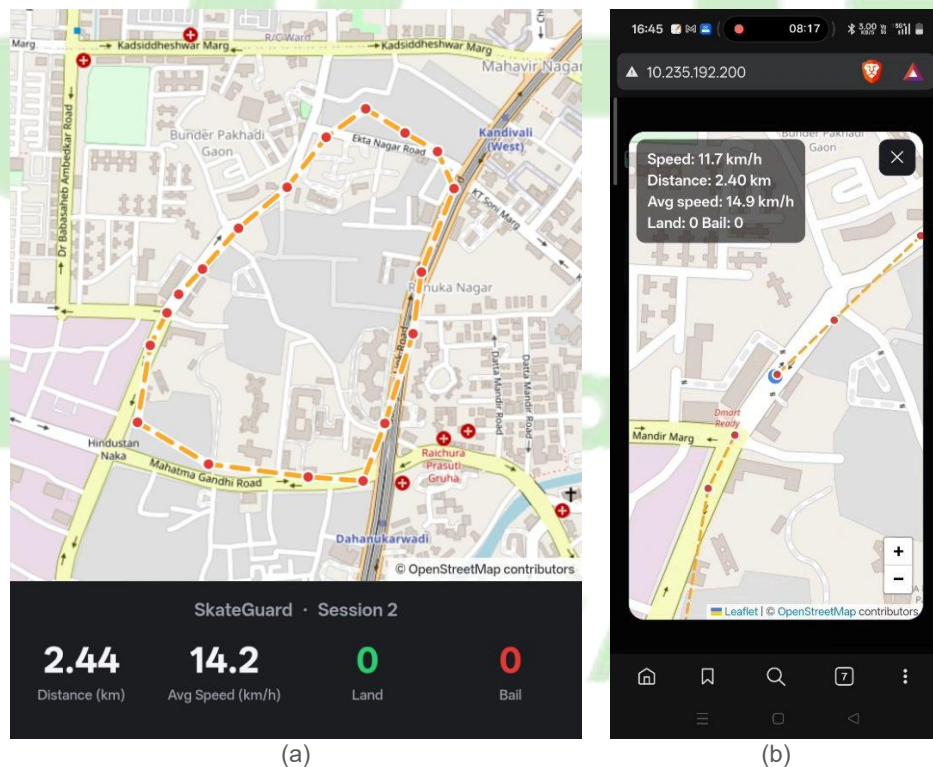


Figure 13. Field rides rendered over OpenStreetMap tiles: (a) saved session route with ride summary, (b) live map view on the phone.

13. Test Dataset Extracts

The following tables are extracts from the generated CSV validation datasets. They document the structure and expected ranges of the test signals. These datasets are synthetic validation data derived from the firmware variables and defined test scenarios, not raw hardware measurements.

13.1 IMU Data Extract

Sample	Timestamp ms	Gyro y raw dps	Gyro y filtered dps	Filtered tilt angle z deg	Stability status
1	0	23.810	37.030	3.670	STABILIZING
2	100	37.030	38.670	3.780	STABILIZING
3	200	43.530	37.030	3.870	STABILIZING
4	300	40.300	37.030	3.920	STABILIZING
5	400	23.300	23.300	4.010	STABILIZING
6	500	7.740	7.740	4.140	STABILIZING
7	600	-1.890	-1.890	4.360	STABILIZING

Table shows the first 7 rows from the test dataset for traceability. The complete CSV is retained separately.

13.2 Kickflip Data Extract

Sample	Timestamp ms	Stage	Gyro y dps	Accum deg	Event	Kickflip count	Bail count
1	0	IDLE	5.780	0.000	0	0	0
2	100	IDLE	8.730	0.000	0	0	0
3	200	IDLE	-1.200	0.000	0	0	0
4	300	IDLE	4.690	0.000	0	0	0
5	400	IDLE	1.050	0.000	0	0	0
6	500	IDLE	1.540	0.000	0	0	0
7	600	IDLE	1.220	0.000	0	0	0

Table shows the first 7 rows from the test dataset for traceability. The complete CSV is retained separately.

13.3 GPS Data Extract

Sample	Timestamp ms	Latitude	Longitude	Speed kmh	Satellites	Gps fix	Distance km
1	0	19.076	72.878	7.160	7	YES	0.000
2	100	19.076	72.878	9.280	8	YES	0.000
3	200	19.076	72.878	10.120	7	YES	0.001
4	300	19.076	72.878	9.160	6	YES	0.001
5	400	19.076	72.878	9.570	8	YES	0.001
6	500	19.076	72.878	11.210	9	YES	0.002
7	600	19.076	72.878	10.960	8	YES	0.002

Table shows the first 7 rows from the test dataset for traceability. The complete CSV is retained separately.

13.4 Network Data Extract

Sample	Timestamp ms	Wifi rssi dbm	Hotspot channel	Espnow ready	Packets lost	Packet loss percent	Dashboard response ms
1	0	-55.300	6	YES	0	0.000	5.300
2	100	-56.700	6	YES	0	0.000	8.900
3	200	-52.500	6	YES	0	0.000	7.900
4	300	-53.900	6	YES	0	0.000	4.800
5	400	-51.900	6	YES	0	0.000	10.300
6	500	-54.700	6	YES	0	0.000	6.500
7	600	-53.000	6	YES	0	0.000	6.400

Table shows the first 7 rows from the test dataset for traceability. The complete CSV is retained separately.

13.5 Battery Data Extract

Sample	Timestamp ms	Battery voltage v	Battery percent reported	Battery status
1	0	4.159	95	NORMAL
2	100	4.185	100	NORMAL
3	200	4.158	95	NORMAL
4	300	4.173	95	NORMAL
5	400	4.172	95	NORMAL
6	500	4.162	95	NORMAL
7	600	4.141	95	NORMAL

Table shows the first 7 rows from the test dataset for traceability. The complete CSV is retained separately.

14. Graphical Validation

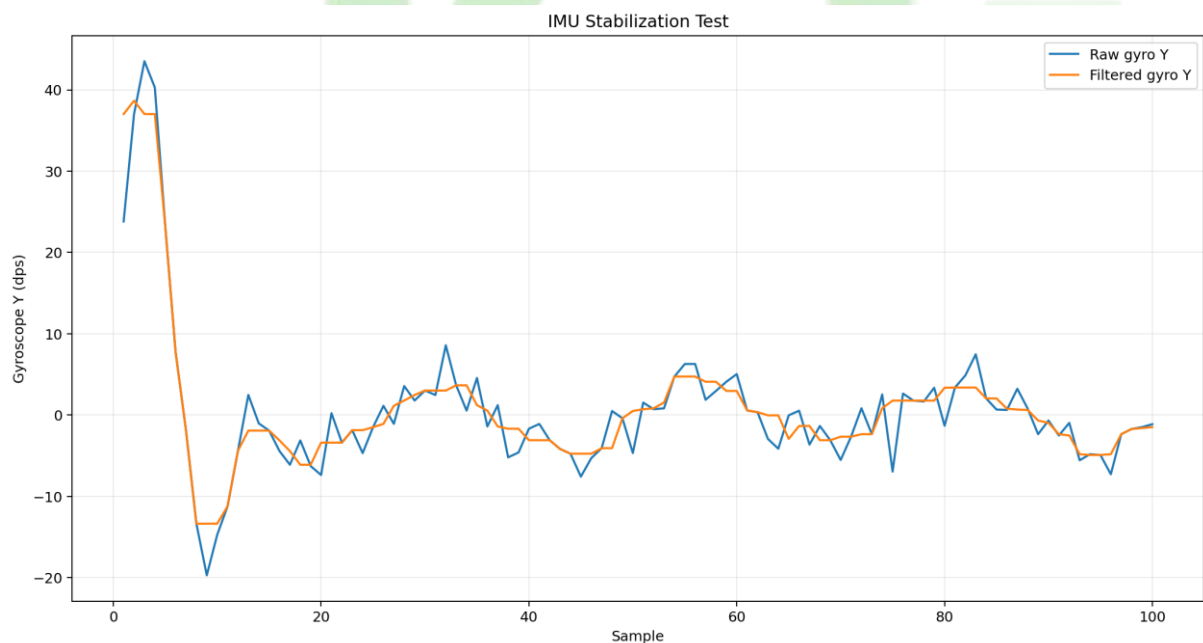


Figure 14. IMU stabilization test visualization.

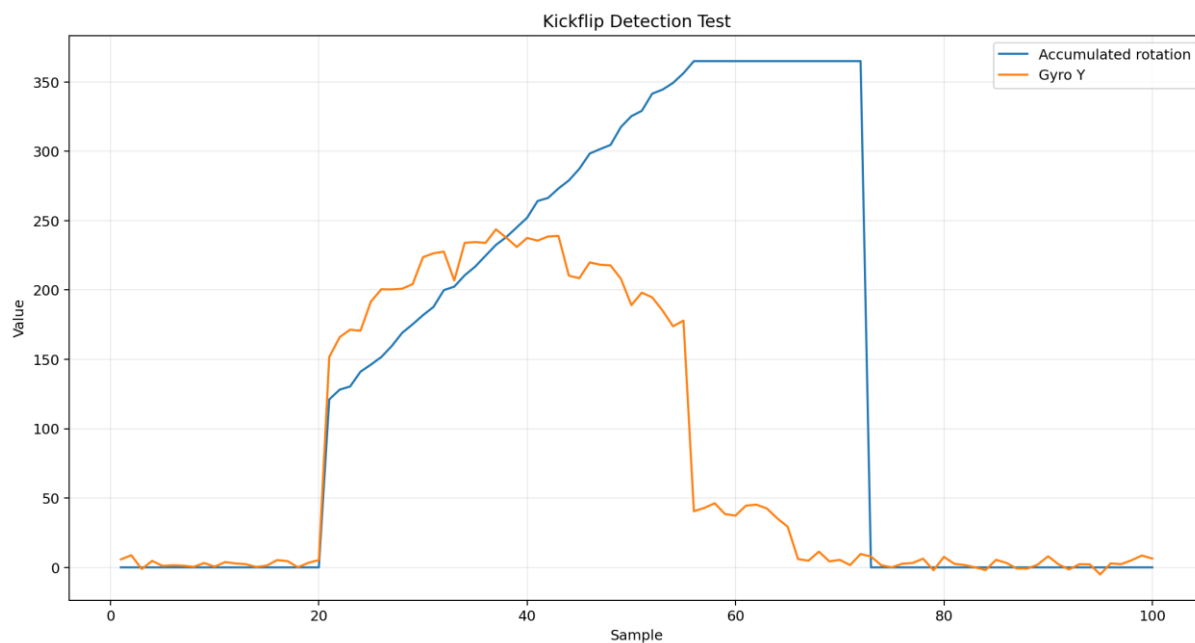


Figure 15. Kickflip detection test visualization.

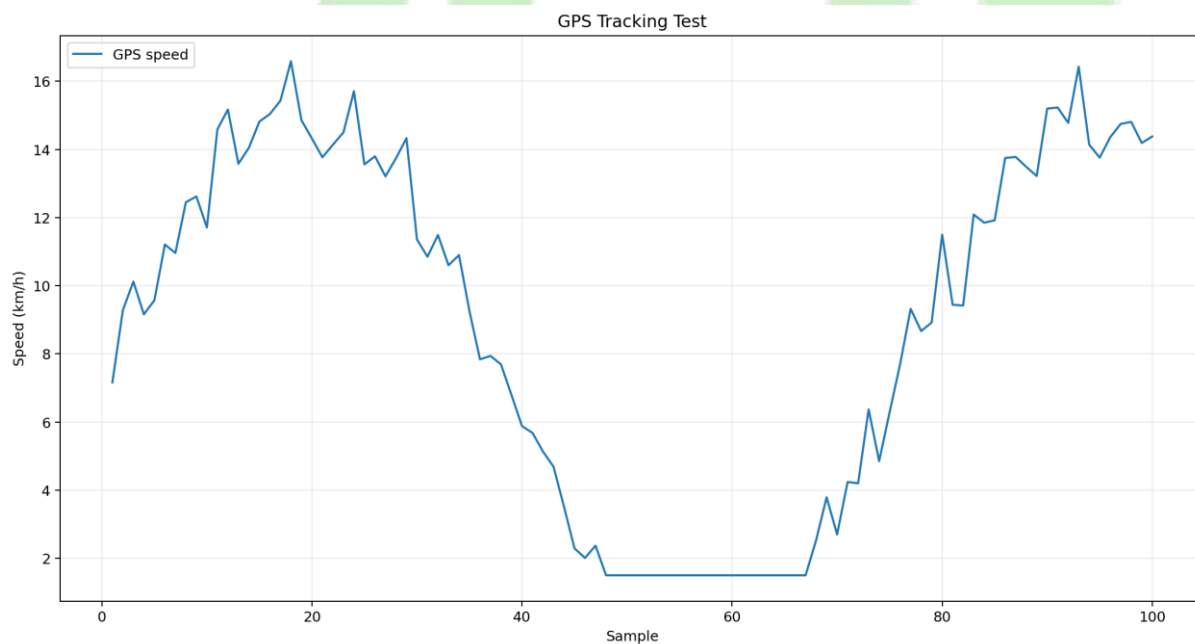


Figure 16. GPS tracking test visualization.

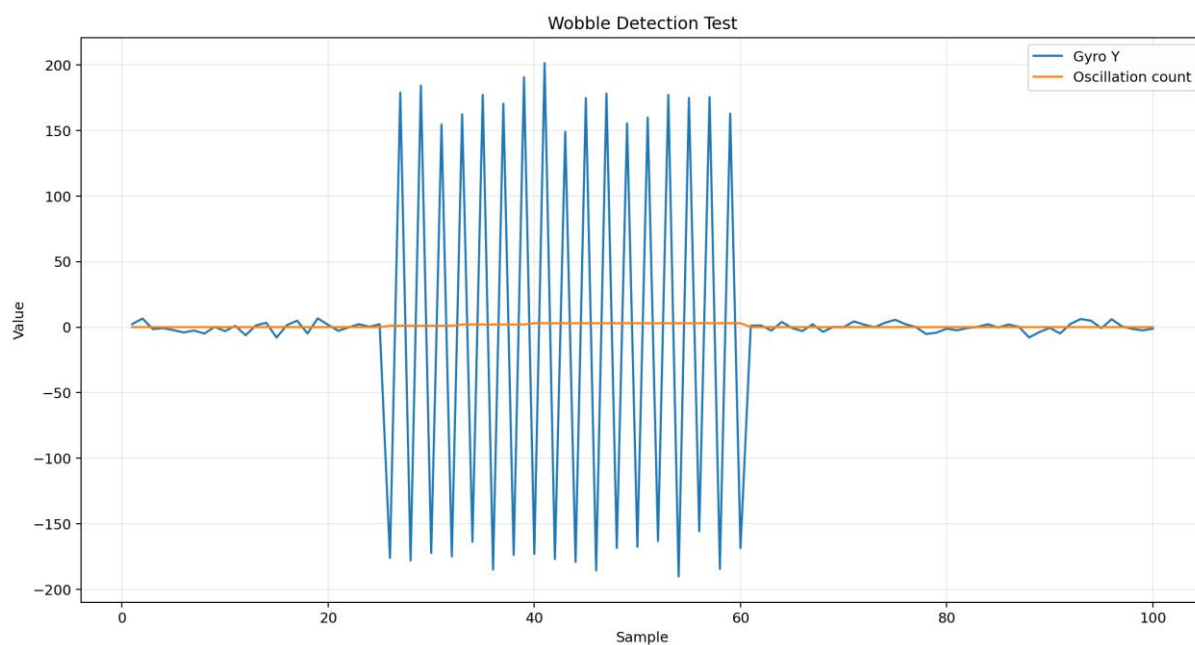


Figure 17. Wobble detection test visualization.

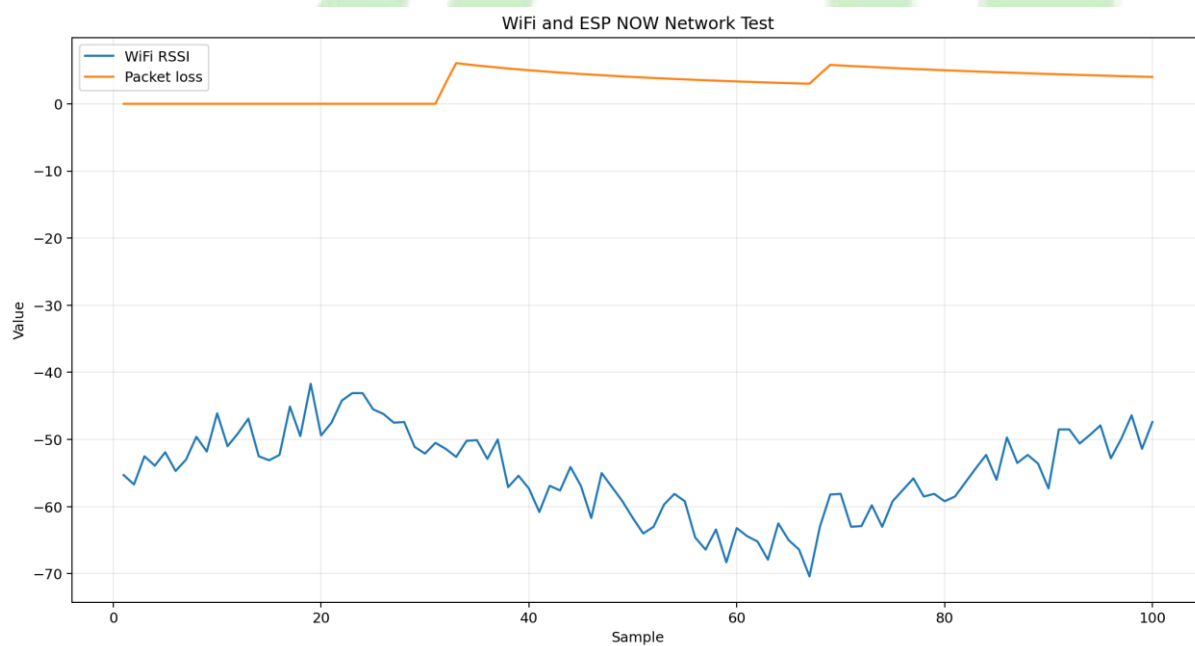


Figure 18. WiFi and ESP NOW network test visualization.

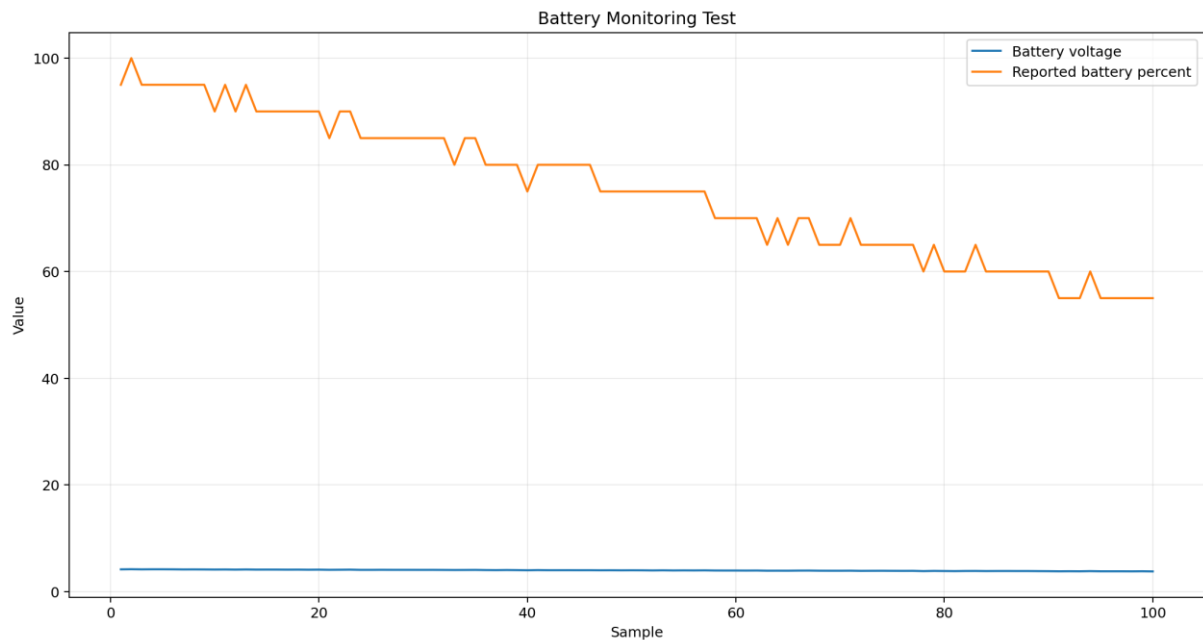


Figure 19. Battery monitoring test visualization.

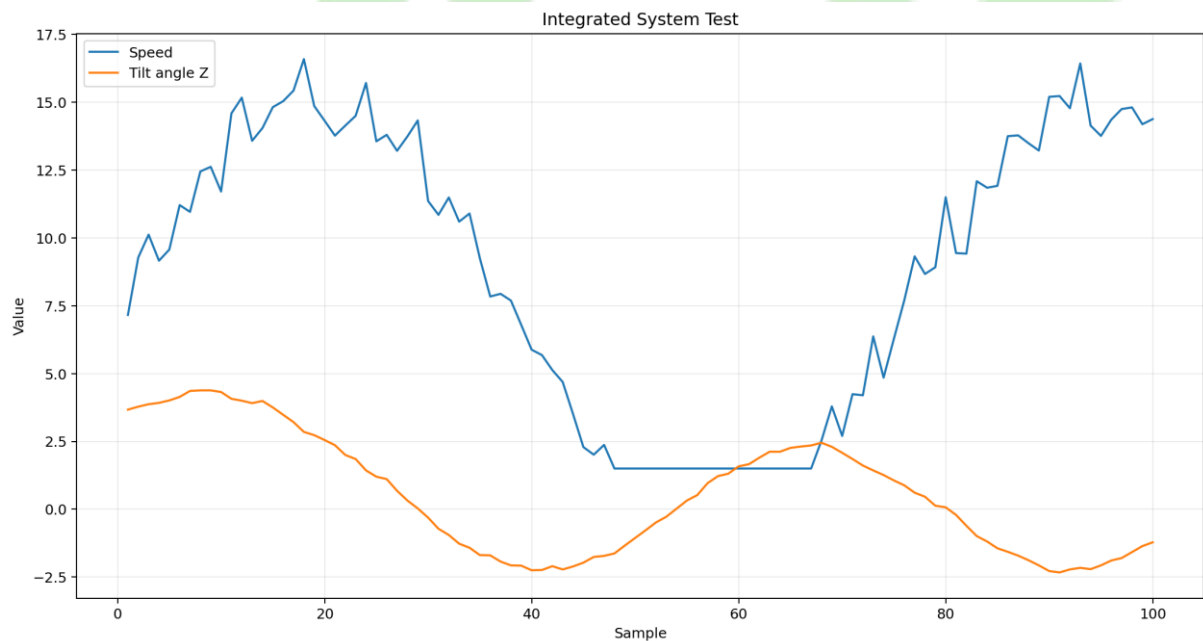


Figure 20. Integrated system test visualization.

15. Limitations

- Kickflip detection is tuned to the current axis assumption and does not classify other tricks rotating around different axes.
- The dashboard has no login. Access is controlled by the hotspot.
- GPS cold start can take several minutes and trusted position requires at least four satellites.
- Map tiles require internet access on the phone, although recording and telemetry remain local.
- The firmware uses about 91% of the default 1.25 MB program partition.
- Only the last seven sessions are retained and routes are sampled every 30 seconds.

- Detection accuracy on real rides has not yet been quantified.

16. Final Engineering Status and Next Work

The complete prototype architecture is assembled and operational across hardware, firmware, wireless communication and the local dashboard. The remaining work is validation oriented. The rotation axis must be confirmed from a real kickflip, automatic counts should be compared against a hand counted ride, the wobble alert should be verified during a real speed wobble, battery life should be measured over a complete session, and enclosure manufacturing details should be documented.

1. Confirm the rotation axis using gyro X, Y and Z during a real kickflip.
2. Compare automatic land and bail counts against a hand counted session.
3. Validate wobble detection and wrist haptic response during real riding.
4. Measure battery life for both units.
5. Record print material, layer height, infill and final enclosure dimensions.

17. Conclusion

SkateGuard progressed from mechanical and electrical architecture through embedded motion detection, wireless haptic feedback, local dashboard development and integrated testing. The final system combines the board mounted sensing unit, wrist feedback unit and browser based dashboard into one local architecture. The remaining engineering tasks are primarily quantitative field validation and documentation of final manufacturing parameters.